CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

OBJECT ORIENTED PROGRAMMING (OOPs)
MINI PROJECT REPORT

Library Management System

Developed in C++ using Object Oriented Programming Concepts

Submitted By	
Name	**Hardeep Singh Sodhi**
Class / Section	**25BCD-2(A)**
UID	**25bcd10215**

aration

I, **Hardeep Singh Sodhi**, a student of class **25BCD-2(A)** with UID **25bcd10215**, hereby declare that the mini project report entitled **"Library Management System"** submitted for the subject Object Oriented Programming (OOPs) is my own work. The program has been designed, written, compiled and tested by me using the C++ programming language.

The concepts of classes, objects, encapsulation, abstraction, inheritance, polymorphism, composition and exception handling have been used in this project as learnt during the course. To the best of my knowledge, this report has not been submitted earlier for any other course or examination.

Signature: _____
Hardeep Singh Sodhi
25BCD-2(A) | UID: 25bcd10215

Date: _____

Acknowledgement

I would like to express my sincere gratitude to my subject teacher for providing guidance and for explaining the fundamentals of Object Oriented Programming in a clear manner. The concepts learnt in the classroom made it possible to design this project in a structured way.

I also thank my university and the department for providing the laboratory facilities and a good learning environment. Finally, I am thankful to my family and friends for their continuous support and encouragement during the development of this project.

Hardeep Singh Sodhi
25BCD-2(A)
UID: 25bcd10215

Abstract

Libraries handle a large number of books and members every day. When records are maintained on paper, tasks such as finding whether a book is available, finding who has taken a book, or calculating a late fine take a lot of time and are prone to mistakes. A computerised Library Management System removes most of this manual effort.

This project is a console-based Library Management System developed in **C++**. It is built completely using Object Oriented Programming. The system stores books and members, allows new books to be added, registers two types of members (Students and Faculty), issues and returns books and calculates the fine automatically if a book is returned late. It also provides a case-insensitive search by title and displays the complete list of books and members.

An abstract base class `Member` defines the common behaviour of all members, and the derived classes `Student` and `Faculty` override its virtual functions to provide their own issue limit, loan period and fine rate. The `Library` class manages all records and works with members through base-class pointers, which is an example of runtime polymorphism. Invalid operations are handled using C++ exception handling.

Keywords: C++, OOPs, Library Management, Inheritance, Polymorphism, Encapsulation, Abstraction, Exception Handling.

Table of Contents

Chapter	Topic
1	Introduction
2	Problem Statement and Scope
3	Objectives
4	System Requirements
5	System Analysis
6	OOPs Concepts Used (with code)
7	Class Design
8	Algorithms and Flow of Control
9	Source Code
10	Sample Output
11	Testing
12	Advantages and Limitations
13	Future Scope
14	Conclusion
15	References

1. Introduction

A library is a collection of books and other resources that are made available to students, teachers and readers. Running a library involves many activities: keeping the list of books, registering members, issuing books, taking them back, checking due dates and collecting fines. If these activities are done by hand, the librarian has to maintain registers and cards, which is slow and creates errors such as lost entries, duplicate entries and wrong fine calculation.

A **Library Management System (LMS)** is a software application that automates these activities. It keeps all the data in one place, allows quick searching and makes sure that the rules of the library are applied in the same way for everyone.

Why Object Oriented Programming?

A library has natural real-world entities such as books, students, faculty and the library itself. Object Oriented Programming allows us to represent each of these entities as a class with its own data and behaviour. This makes the program easier to understand, easier to maintain and easy to extend. For example, a new type of member can be added without rewriting the existing code. Because of this, OOPs is a very suitable approach for a Library Management System.

Overview of the Project

- **Language:** C++ (C++11 standard).
- **Type:** Console (text) based, menu driven application.

- **Members supported:** Students and Faculty, each with different rules.
- **Main operations:** add book, register member, issue book, return book with fine, search book, display books and members.
- **Safety:** all invalid inputs and operations are handled through exceptions and error messages.

2. Problem Statement and Scope

Problem Statement

To design and develop a Library Management System in C++ that manages the records of books and members and automates the issue and return process, using the concepts of Object Oriented Programming, so that the work of the librarian becomes faster and more accurate.

Problems in the Manual System

- Searching for a book in registers takes a long time.
- It is difficult to know whether a book is currently available or issued.
- The librarian must remember or check the issue limit of each member.
- Fine calculation by hand can be wrong.
- Records can be damaged, lost or duplicated.

Scope of the Project

The project covers the core functions of a small library:

- Maintaining a list of books with unique IDs.
- Maintaining members of two categories with unique IDs.
- Issuing a book only if it exists, is available and the member has not crossed his or her limit.
- Returning a book and calculating a late fine according to the type of the member.
- Searching and displaying records.

Out of Scope

Permanent storage in files or databases, graphical user interface, online access and real calendar dates are not part of this version. They are listed in the Future Scope chapter.

3. Objectives

- To apply the main OOPs concepts (class, object, encapsulation, abstraction, inheritance and polymorphism) to a real-life problem.
- To store and manage the records of books and members using classes and the STL `vector` container.
- To automate the issue and return of books along with the calculation of fine.
- To use abstract classes and virtual functions so that different member types follow different rules through one common interface.
- To use dynamic memory allocation safely with proper destructors so that there is no memory leak.
- To use exception handling for reporting errors such as duplicate IDs, missing records and exceeded limits.
- To develop a program which is modular, readable and easy to extend.

ess Rules Implemented

Rule	Student	Faculty
Maximum books at a time	3	5
Loan period	14 days	30 days
Fine per late day	Rs. 2	Rs. 1
Fine formula	(days kept - loan days) x fine per day	(days kept - loan days) x fine per day

4. System Requirements

Hardware Requirements

Component	Minimum Requirement
Processor	Any dual-core processor (1 GHz or above)
RAM	512 MB or more
Hard disk	50 MB free space
Display / Input	Monitor and keyboard

Software Requirements

Item	Details
Operating system	Windows / Linux / macOS
Language	C++ (C++11 or later)
Compiler	g++ (GCC), MinGW, or Visual Studio C++ compiler
Editor / IDE	Code::Blocks, Visual Studio Code, Dev-C++ or any text editor

Header Files and Libraries Used

Header	Purpose
<iostream>	Input and output using cin and cout
<vector>	Dynamic arrays to store books and members
<string>	String handling for names, titles and authors
<iomanip>	setw() for neat tabular output
<stdexcept>	runtime_error class for exception handling
<algorithm>	transform() for case-insensitive search

How to Compile and Run

```
g++ -std=c++11 library.cpp -o library
./library          (Linux / macOS)
library.exe        (Windows)
```

Functional Requirements

ID	Requirement
FR1	The system shall allow the librarian to add a book with ID, title and author.
FR2	The system shall register a Student or Faculty member with ID and name.
FR3	The system shall issue an available book to a registered member within his or her issue limit.
FR4	The system shall accept a returned book and calculate the late fine, if any.
FR5	The system shall search books by a keyword in the title.
FR6	The system shall display all books with status and all members with issued count.
FR7	The system shall display a proper error message for every invalid operation.

Non-Functional Requirements

- **Usability:** simple numbered menu that can be used without training.
- **Reliability:** the program must not crash on wrong IDs or duplicate entries.
- **Maintainability:** every responsibility is placed in a separate class.
- **Extensibility:** new member types can be added by deriving a new class from Member.
- **Performance:** operations are instantaneous for the small data size handled.

Actors and Use Cases

The only actor in this system is the **Librarian**, who operates the menu. The use cases are shown below.

Use Case	Description	Result
Add Book	Librarian enters ID, title and author.	Book stored as Available
Register Member	Librarian selects member type and enters ID and name.	Student or Faculty object created
Issue Book	Librarian enters Book ID and Member ID.	Book marked Issued, member count + 1
Return Book	Librarian enters Book ID and number of days kept.	Book marked Available, fine shown
Search Book	Librarian enters a keyword.	Matching books listed
View Records	Librarian selects display option.	Books or members shown in table

6. OOPs Concepts Used

The following table gives a quick summary of the concepts. Each concept is then explained with the related code from the project.

Concept	Where it is used
Class and Object	Book, Member, Student, Faculty, Library

...t	Where it is used
Encapsulation	Private data of Book and Library
Abstraction	Abstract class Member with pure virtual functions
Inheritance	Student and Faculty derive from Member
Polymorphism	Virtual functions called through Member*
Composition	Library contains Books and Members
Constructor / Destructor	Initialisation and memory clean-up
Exception Handling	try / catch with runtime_error

6.1 Class and Object

A class is a blueprint which combines data and the functions that work on that data. An object is a real instance of a class. In this project every book is an object of class `Book`, and every registered person is an object of `Student` or `Faculty`.

```
Book b(101, "Let Us C++", "Yashavant Kanetkar");   // object of class Book
Member* m = new Student(1, "Hardeep Singh Sodhi");  // object of class Student
```

6.2 Encapsulation

Encapsulation means wrapping data and functions together and hiding the data from direct outside access. In the `Book` class all data members are *private*. They can be read only through getter functions and changed only through the controlled functions `issue()` and `giveBack()`. This guarantees that the status and the 'issued to' fields always stay consistent.

```
class Book {
private:
    int id;  string title, author;
    bool available;  int issuedTo;
public:
    void issue(int memberId) { available = false; issuedTo = memberId; }
    void giveBack()          { available = true;  issuedTo = -1; }
};
```

6.3 Abstraction

Abstraction shows only the essential interface and hides the details. `Member` is an **abstract class** because it contains pure virtual functions. We cannot create an object of Member; it only tells what every kind of member must provide: its type, issue limit, loan period and fine rate. How these are decided is left to the derived classes.

```
virtual string getType()   const = 0;
virtual int    maxBooks()   const = 0;
virtual int    loanDays()   const = 0;
virtual double finePerDay() const = 0;
```

6.4 Inheritance

Inheritance allows a class to acquire the properties of another class. `Student` and `Faculty` inherit id, name and issuedCount and the common functions (canIssue, display, etc.) from `Member`. This avoids repeating the same code and models the real-world relationship 'a Student *is a* Member'.

```
class Student : public Member { ... };
class Faculty : public Member { ... };
```

lymorphism

Polymorphism means 'one interface, many forms'. The `Library` stores all members in a `vector<Member*>`. When it calls `m->maxBooks()` or `m->finePerDay()`, the correct version (Student's or Faculty's) is selected automatically at run time. Because of this, the Library class does not need any *if (type == Student)* checks.

```
class Student : public Member {
    int    maxBooks()   const override { return 3; }
    double finePerDay() const override { return 2.0; }
};
class Faculty : public Member {
    int    maxBooks()   const override { return 5; }
    double finePerDay() const override { return 1.0; }
};

if (!m->canIssue()) throw runtime_error("Issue limit reached for this member!");
```

6.6 Composition (Has-A Relationship)

The `Library` class *has* a collection of books and a collection of members. Books are stored by value inside `vector<Book>` and members are stored as pointers inside `vector<Member*>` (pointers are needed because of polymorphism).

6.7 Constructors and Destructors

Constructors initialise the data members when an object is created, for example a new Book is always created as available and not issued to anybody. Members are created with `new`, so the `Library` destructor deletes each of them. The base class destructor is declared **virtual** so that the derived object is destroyed properly when deleted through a base-class pointer.

```
virtual ~Member() {}
~Library() { for (auto* m : members) delete m; }
```

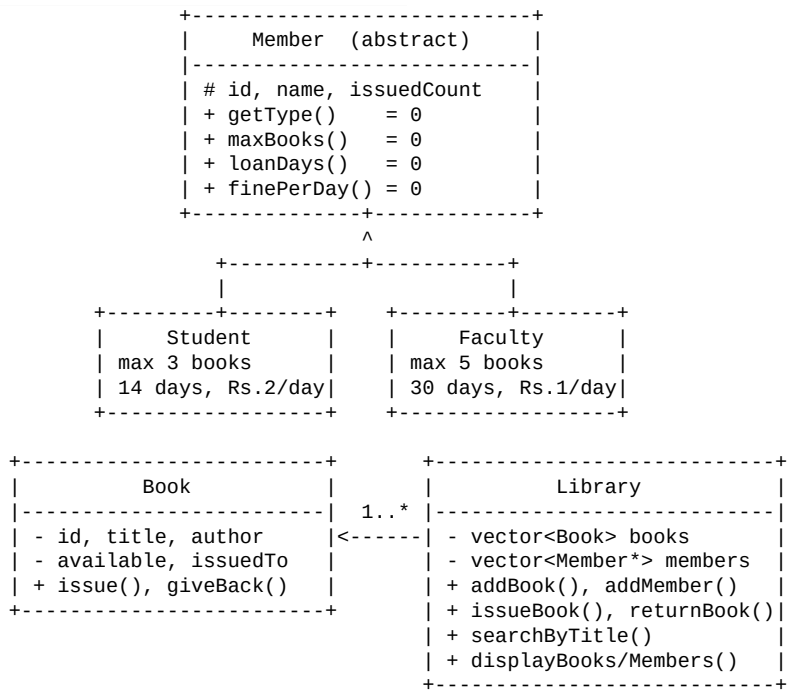
6.8 Exception Handling

Instead of printing messages inside deep functions or crashing, the Library functions `throw` a `runtime_error` with a clear message. The menu loop in `main()` catches it and prints the message, so the program continues to run.

```
try {
    lib.issueBook(id, mid);
} catch (const exception& e) {
    cout << "Error: " << e.what() << endl;
}
```

7. Class Design

Class Diagram



The arrow with a hollow head shows inheritance (Student and Faculty are derived from Member). The Library class is associated with many books and many members through composition.

7.2 Class Member (abstract base class)

Data Member	Type	Access	Purpose
id	int	protected	Unique member ID
name	string	protected	Name of the member
issuedCount	int	protected	Number of books currently issued

Member Function	Purpose
getType() = 0	Returns "Student" or "Faculty" (pure virtual)
maxBooks() = 0	Maximum books allowed (pure virtual)
loanDays() = 0	Number of days a book can be kept without fine (pure virtual)
finePerDay() = 0	Fine in rupees for each late day (pure virtual)
canIssue()	Returns true if issuedCount is less than maxBooks()
incrementIssued() / decrementIssued()	Updates the issued count when a book is issued or returned
display()	Prints one row of the member table
getId(), getName(), getIssuedCount()	Getter functions

7.3 Classes Student and Faculty

Both classes have only a constructor and the four overridden functions. They show how the same interface behaves differently.

n	Student	Faculty
getType()	"Student"	"Faculty"
maxBooks()	3	5
loanDays()	14	30
finePerDay()	2.0	1.0

7.4 Class Book

Data Member	Type	Access	Purpose
id	int	private	Unique book ID
title	string	private	Title of the book
author	string	private	Author name
available	bool	private	true if the book is on the shelf
issuedTo	int	private	Member ID holding the book, -1 if none

Member Function	Purpose
Book(id, title, author)	Constructor; sets available = true and issuedTo = -1
issue(memberId)	Marks the book as issued to the given member
giveBack()	Marks the book as available again
isAvailable(), getIssuedTo()	Return the current status
getId(), getTitle(), getAuthor()	Getter functions
display()	Prints one row of the book table

Member	Type / Access	Purpose
books	vector<Book> (private)	All books of the library
members	vector<Member*> (private)	All registered members
findBook(id)	private function	Returns pointer to the book or nullptr
findMember(id)	private function	Returns pointer to the member or nullptr
addBook()	public	Adds a book after checking for duplicate ID
addMember()	public	Registers a member after checking for duplicate ID
issueBook()	public	Validates and issues a book
returnBook()	public	Takes back a book and calculates the fine
searchByTitle()	public	Case-insensitive partial search
displayBooks() / displayMembers()	public	Print the records in table form
~Library()	destructor	Deletes all dynamically created members

8. Algorithms and Flow of Control

8.1 Main Menu

```
1. Add Book            5. Return Book
2. Register Student    6. Search Book
3. Register Faculty    7. Show All Books
4. Issue Book          8. Show All Members
                       0. Exit
```

The menu is shown inside a *do-while* loop. A *switch* statement calls the required function and a *try-catch* block around it displays any error. The loop ends when the user enters 0.

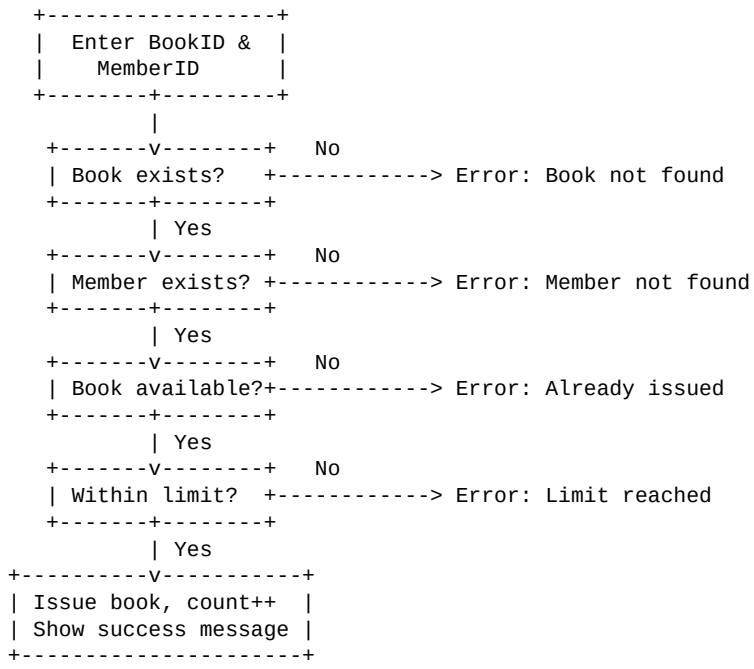
8.2 Algorithm: Issue Book

```
1. Read bookId and memberId.
2. Find the book; if not found -> throw "Book not found!"
3. Find the member; if not found -> throw "Member not found!"
4. If the book is not available -> throw "Book is already issued!"
5. If member.canIssue() is false -> throw "Issue limit reached for this member!"
6. Mark the book as issued to memberId.
7. Increase the member's issuedCount by 1.
8. Display "issued" message with the loan period of that member type.
```

8.3 Algorithm: Return Book

```
1. Read bookId and daysKept.
2. Find the book; if not found -> throw "Book not found!"
3. If the book is already available -> throw "This book was not issued!"
4. Find the member who holds the book.
5. late = daysKept - member.loanDays()
6. If late > 0 then fine = late x member.finePerDay() else fine = 0
7. Mark the book as available; decrease member's issuedCount by 1.
8. Display the return message and the fine, if any.
```

8.4 Flowchart: Issue Book



8.5 Algorithm: Search Book

```
1. Read keyword; convert it to lower case.
2. For every book in the list:
      a. Convert the title to lower case.
      b. If the title contains the keyword -> display the book, found = true.
3. If found is false -> display "No matching book found."
```

8.6 Fine Calculation Example

Member	Days kept	Loan period	Late days	Fine
Student	20	14	6	6 x 2 = Rs. 12
Faculty	40	30	10	10 x 1 = Rs. 10
Faculty	25	30	0	Rs. 0

Source Code (library.cpp)

```
/*
 * Library Management System - OOPs Mini Project (C++)
 * Name  : Hardeep Singh Sodhi
 * Class : 25BCD-2(A)
 * UID   : 25bcd10215
 */
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <stdexcept>
#include <algorithm>
using namespace std;

// ---------- Abstract base class (Abstraction) ----------
class Member {
protected:
    int id;
    string name;
    int issuedCount;
public:
    Member(int id, const string& name) : id(id), name(name), issuedCount(0) {}
    virtual ~Member() {}

    // Pure virtual functions -> Polymorphism
    virtual string getType() const = 0;
    virtual int    maxBooks() const = 0;
    virtual int    loanDays() const = 0;
    virtual double finePerDay() const = 0;

    int getId() const { return id; }
    string getName() const { return name; }
    int getIssuedCount() const { return issuedCount; }
    bool canIssue() const { return issuedCount < maxBooks(); }
    void incrementIssued() { issuedCount++; }
    void decrementIssued() { if (issuedCount > 0) issuedCount--; }

    void display() const {
        cout << left << setw(6) << id << setw(20) << name << setw(10) << getType()
             << (to_string(issuedCount) + "/" + to_string(maxBooks())) << endl;
    }
};

// ---------- Inheritance ----------
class Student : public Member {
public:
    Student(int id, const string& n) : Member(id, n) {}
    string getType() const override { return "Student"; }
    int maxBooks() const override { return 3; }
    int loanDays() const override { return 14; }
    double finePerDay() const override { return 2.0; }
};

class Faculty : public Member {
public:
    Faculty(int id, const string& n) : Member(id, n) {}
    string getType() const override { return "Faculty"; }
    int maxBooks() const override { return 5; }
    int loanDays() const override { return 30; }
    double finePerDay() const override { return 1.0; }
};

// ---------- Book class (Encapsulation) ----------
class Book {
private:
    int id;
    string title, author;
    bool available;
    int issuedTo;                // member id, -1 if not issued
public:
    Book(int id, const string& t, const string& a)
        : id(id), title(t), author(a), available(true), issuedTo(-1) {}

    int getId() const { return id; }
    string getTitle() const { return title; }
    string getAuthor() const { return author; }
```

```
    bool isAvailable() const { return available; }
    int getIssuedTo() const { return issuedTo; }

    void issue(int memberId) { available = false; issuedTo = memberId; }
    void giveBack() { available = true; issuedTo = -1; }

    void display() const {
        cout << left << setw(6) << id << setw(28) << title << setw(20) << author
             << (available ? "Available" : "Issued") << endl;
    }
};

// ---------- Library class (Composition) ----------
class Library {
private:
    vector<Book> books;
    vector<Member*> members;

    Book* findBook(int id) {
        for (auto& b : books) if (b.getId() == id) return &b;
        return nullptr;
    }
    Member* findMember(int id) {
        for (auto* m : members) if (m->getId() == id) return m;
        return nullptr;
    }
public:
    ~Library() { for (auto* m : members) delete m; }

    void addBook(int id, const string& t, const string& a) {
        if (findBook(id)) throw runtime_error("Book ID already exists!");
        books.push_back(Book(id, t, a));
        cout << "Book added successfully.\n";
    }

    void addMember(Member* m) {
        if (findMember(m->getId())) {
            delete m;
            throw runtime_error("Member ID already exists!");
        }
        members.push_back(m);
        cout << m->getType() << " registered successfully.\n";
    }

    void issueBook(int bookId, int memberId) {
        Book* b = findBook(bookId);
        Member* m = findMember(memberId);
        if (!b) throw runtime_error("Book not found!");
        if (!m) throw runtime_error("Member not found!");
        if (!b->isAvailable()) throw runtime_error("Book is already issued!");
        if (!m->canIssue()) throw runtime_error("Issue limit reached for this member!");
        b->issue(memberId);
        m->incrementIssued();
        cout << "\"" << b->getTitle() << "\" issued to " << m->getName()
             << ". Return within " << m->loanDays() << " days.\n";
    }

    void returnBook(int bookId, int daysKept) {
        Book* b = findBook(bookId);
        if (!b) throw runtime_error("Book not found!");
        if (b->isAvailable()) throw runtime_error("This book was not issued!");
        Member* m = findMember(b->getIssuedTo());
        int late = daysKept - m->loanDays();
        double fine = (late > 0) ? late * m->finePerDay() : 0;
        b->giveBack();
        m->decrementIssued();
        cout << "\"" << b->getTitle() << "\" returned by " << m->getName() << ".\n";
        if (fine > 0) cout << "Late by " << late << " day(s). Fine = Rs. " << fine << endl;
        else          cout << "No fine. Returned on time.\n";
    }

    void searchByTitle(const string& key) const {
        bool found = false;
        string k = key;
        transform(k.begin(), k.end(), k.begin(), ::tolower);
        for (const auto& b : books) {
            string t = b.getTitle();
            transform(t.begin(), t.end(), t.begin(), ::tolower);
            if (t.find(k) != string::npos) { b.display(); found = true; }
```

```
    }
    if (!found) cout << "No matching book found.\n";
    }

    void displayBooks() const {
        cout << left << setw(6) << "ID" << setw(28) << "Title" << setw(20) << "Author" << "Status\n";
        cout << string(66, '-') << endl;
        for (const auto& b : books) b.display();
    }

    void displayMembers() const {
        cout << left << setw(6) << "ID" << setw(20) << "Name" << setw(10) << "Type" << "Issued\n";
        cout << string(46, '-') << endl;
        for (const auto* m : members) m->display();
    }
};

// ---------- main ----------
int main() {
    Library lib;
    int choice;
    cout << "===== LIBRARY MANAGEMENT SYSTEM =====\n";
    do {
        cout << "\n1. Add Book\n2. Register Student\n3. Register Faculty\n4. Issue Book\n"
             << "5. Return Book\n6. Search Book\n7. Show All Books\n8. Show All Members\n0. Exit\n"
             << "Enter choice: ";
        cin >> choice;
        try {
            int id, mid, days; string t, a, n;
            switch (choice) {
            case 1:
                cout << "Book ID: "; cin >> id; cin.ignore();
                cout << "Title: ";   getline(cin, t);
                cout << "Author: ";  getline(cin, a);
                lib.addBook(id, t, a); break;
            case 2: case 3:
                cout << "Member ID: "; cin >> id; cin.ignore();
                cout << "Name: "; getline(cin, n);
                if (choice == 2) lib.addMember(new Student(id, n));
                else             lib.addMember(new Faculty(id, n));
                break;
            case 4:
                cout << "Book ID: "; cin >> id;
                cout << "Member ID: "; cin >> mid;
                lib.issueBook(id, mid); break;
            case 5:
                cout << "Book ID: "; cin >> id;
                cout << "Days kept: "; cin >> days;
                lib.returnBook(id, days); break;
            case 6:
                cout << "Enter title keyword: "; cin.ignore(); getline(cin, t);
                lib.searchByTitle(t); break;
            case 7: lib.displayBooks(); break;
            case 8: lib.displayMembers(); break;
            case 0: cout << "Thank you for using the system!\n"; break;
            default: cout << "Invalid choice!\n";
            }
        } catch (const exception& e) {
            cout << "Error: " << e.what() << endl;
        }
    } while (choice != 0);
    return 0;
}
```

Sample Output

Compiled with `g++ -std=c++11 library.cpp -o library`. In the sessions below, the menu text is abbreviated as [MENU] and the values typed by the user follow each prompt.

10.1 Session 1: Adding records, issuing and returning a late book

```
===== LIBRARY MANAGEMENT SYSTEM =====
[MENU]  Enter choice: 1
Book ID: 101
Title: Let Us C++
Author: Yashavant Kanetkar
Book added successfully.

[MENU]  Enter choice: 1
Book ID: 102
Title: Data Structures in C
Author: Reema Thareja
Book added successfully.

[MENU]  Enter choice: 1
Book ID: 103
Title: Object Oriented Programming
Author: E. Balagurusamy
Book added successfully.

[MENU]  Enter choice: 2
Member ID: 1
Name: Hardeep Singh Sodhi
Student registered successfully.

[MENU]  Enter choice: 3
Member ID: 2
Name: Dr. Anil Sharma
Faculty registered successfully.

[MENU]  Enter choice: 7
ID    Title                       Author              Status
-----------------------------------------------------------------
101   Let Us C++                  Yashavant Kanetkar  Available
102   Data Structures in C        Reema Thareja       Available
103   Object Oriented Programming E. Balagurusamy     Available

[MENU]  Enter choice: 8
ID    Name                Type      Issued
-----------------------------------------------
1     Hardeep Singh Sodhi Student   0/3
2     Dr. Anil Sharma     Faculty   0/5

[MENU]  Enter choice: 4
Book ID: 101
Member ID: 1
"Let Us C++" issued to Hardeep Singh Sodhi. Return within 14 days.

[MENU]  Enter choice: 4
Book ID: 101
Member ID: 2
Error: Book is already issued!

[MENU]  Enter choice: 6
Enter title keyword: Object
103   Object Oriented Programming E. Balagurusamy     Available

[MENU]  Enter choice: 5
Book ID: 101
Days kept: 20
"Let Us C++" returned by Hardeep Singh Sodhi.
Late by 6 day(s). Fine = Rs. 12

[MENU]  Enter choice: 5
Book ID: 101
Days kept: 10
Error: This book was not issued!

[MENU]  Enter choice: 4
```

```
Book ID: 999
Member ID: 1
Error: Book not found!

[MENU]  Enter choice: 0
Thank you for using the system!
```

10.2 Session 2: Issue limit, duplicate IDs, fine and search

```
[MENU]  Enter choice: 1      (Books 201 to 205 added successfully)
...
[MENU]  Enter choice: 1
Book ID: 201
Title: Duplicate Book
Author: Someone
Error: Book ID already exists!

[MENU]  Enter choice: 2
Member ID: 11
Name: Riya Verma
Student registered successfully.

[MENU]  Enter choice: 2
Member ID: 11
Name: Another Student
Error: Member ID already exists!

[MENU]  Enter choice: 4     (Book 201, Member 11)
"Python Basics" issued to Riya Verma. Return within 14 days.
[MENU]  Enter choice: 4     (Book 202, Member 11)
"Java Complete Reference" issued to Riya Verma. Return within 14 days.
[MENU]  Enter choice: 4     (Book 203, Member 11)
"Operating System Concepts" issued to Riya Verma. Return within 14 days.
[MENU]  Enter choice: 4     (Book 204, Member 11)
Error: Issue limit reached for this member!

[MENU]  Enter choice: 8
ID    Name                Type      Issued
---------------------------------------------
11    Riya Verma          Student   3/3

[MENU]  Enter choice: 5     (Book 203, Days kept 14)
"Operating System Concepts" returned by Riya Verma.
No fine. Returned on time.

[MENU]  Enter choice: 5     (Book 202, Days kept 16)
"Java Complete Reference" returned by Riya Verma.
Late by 2 day(s). Fine = Rs. 4

[MENU]  Enter choice: 7
ID    Title                      Author              Status
-----------------------------------------------------------------
201   Python Basics              Guido Rossum        Issued
202   Java Complete Reference    Herbert Schildt     Available
203   Operating System Concepts  Galvin              Available
204   Computer Networks          Tanenbaum           Available
205   Discrete Mathematics       Kenneth Rosen       Available

[MENU]  Enter choice: 6     (keyword: computer)
204   Computer Networks          Tanenbaum           Available
[MENU]  Enter choice: 6     (keyword: xyz)
No matching book found.
[MENU]  Enter choice: 4     (Book 205, Member 77)
Error: Member not found!
[MENU]  Enter choice: 9
Invalid choice!
[MENU]  Enter choice: 0
Thank you for using the system!
```

Testing

The program was tested with valid and invalid inputs. Black-box testing was used: for each test case the input was given and the actual output was compared with the expected output. All test cases passed.

TC	Test Description	Input	Expected Result	Actual Result	Status
1	Add a new book	ID 101, Let Us C++	Book added	Book added successfully.	Pass
2	Add book with duplicate ID	ID 201 again	Error message	Error: Book ID already exists!	Pass
3	Register a student	ID 1, Hardeep Singh Sodhi	Student registered	Student registered successfully.	Pass
4	Register faculty	ID 2, Dr. Anil Sharma	Faculty registered	Faculty registered successfully.	Pass
5	Duplicate member ID	ID 11 again	Error message	Error: Member ID already exists!	Pass
6	Issue available book	Book 101, Member 1	Book issued, 14 days	Issued ... Return within 14 days.	Pass
7	Issue already issued book	Book 101, Member 2	Error message	Error: Book is already issued!	Pass
8	Issue to member over limit	4th book, Student	Error message	Error: Issue limit reached for this member!	Pass
9	Issue non-existing book	Book 999	Error message	Error: Book not found!	Pass
10	Issue to non-existing member	Member 77	Error message	Error: Member not found!	Pass
11	Return on time	14 days (Student)	No fine	No fine. Returned on time.	Pass
12	Return late (Student)	20 days	Fine Rs. 12	Late by 6 day(s). Fine = Rs. 12	Pass
13	Return late (Student)	16 days	Fine Rs. 4	Late by 2 day(s). Fine = Rs. 4	Pass
14	Return a book not issued	Book 101	Error message	Error: This book was not issued!	Pass
15	Search with partial keyword	computer	Computer Networks	204 Computer Networks ...	Pass
16	Search with no match	xyz	No book found	No matching book found.	Pass
17	Invalid menu choice	9	Invalid message	Invalid choice!	Pass

	Test Description	Input	Expected Result	Actual Result	Status
18	Exit	0	Program ends	Thank you for using the system!	Pass

Conclusion of Testing

The program handled all valid and invalid cases correctly. The system never terminated abnormally because every error is reported through exception handling and the menu continues to work.

12. Advantages and Limitations

Advantages

- Modular design: each class has a single, clear responsibility.
- Code reuse through inheritance of common member behaviour.
- New member types can be added without changing the Library class (open for extension).
- Fine and limit rules are automatically applied through polymorphism.
- Safe error handling; the program does not crash on wrong input of IDs.
- No memory leak because all dynamically created members are deleted in the destructor.
- Simple menu interface that is easy for the librarian to use.

Limitations

- Data is stored in memory only and is lost when the program is closed.
- The number of days kept is entered manually; real dates are not used.
- Only one librarian (single user) can use the system.
- A member's issued books are counted but the list of titles is not stored in the Member object.
- Console interface only; there is no graphical user interface.
- If a non-numeric value is typed where a number is expected, the input stream is not reset (can be improved with input validation).

13. Future Scope

The project is designed in a way that new features can be added easily. Some possible improvements are listed below.

Enhancement	How it can be done
Permanent data storage	Use file handling (ifstream / ofstream) or a database such as MySQL or SQLite to save books and members.
Real dates	Store issue date and due date using <ctime> / <chrono> and calculate late days automatically.
New member types	Derive classes such as ResearchScholar or Guest from Member with their own rules.
Book reservation	Add a queue of members waiting for an issued book.
Login system	Add Admin and Librarian classes with username and password.
Delete / update records	Add functions to remove or edit books and members.

ement	How it can be done
Faster search	Use unordered_map for O(1) search by ID.
Input validation	Check cin failure state and clear the buffer for wrong data types.
Graphical interface	Develop a GUI using Qt or move to a web based version.
Reports	Show list of defaulters, most issued books and total fine collected.

14. Conclusion

The Library Management System developed in this mini project successfully manages books and members, issues and returns books and calculates fines according to the type of the member. The project made use of all the important concepts of Object Oriented Programming: classes and objects, encapsulation, abstraction, inheritance, polymorphism, composition, constructors and destructors and exception handling.

Through this project I understood how a real-world problem can be divided into classes and how virtual functions make a program flexible. The system was tested with eighteen test cases and gave the correct output in all of them. With the enhancements listed in the future scope, it can be developed into a complete library software.

15. References

- E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw Hill Education.
- Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.
- Herbert Schildt, *C++: The Complete Reference*, McGraw Hill.
- Robert Lafore, *Object-Oriented Programming in C++*, Sams Publishing.
- cppreference.com - C++ reference documentation (https://en.cppreference.com).
- Class notes and lecture material of the subject Object Oriented Programming.